



Introduction to Software Engineering

Technical Report #2 for "FreeBeats" project:

Team members:

1. Abdelrahman Hossam 23-101130 (Leader)
2. Arwa Mahmoud 23-10131
3. Rodaina Al-dahshan 23-101193
4. Hana Shoaib 23-101166
5. Mohamed Fathy 23-101167

Event Decomposition Technique:

We began the analysis phase by applying the **Event Composition Technique** to identify all events that trigger system behavior.

This allowed us to capture a complete picture of what initiates system actions, whether the trigger comes from:

- an external user,
- a time-based schedule,
- or a change in the internal state of the system.

By organizing these events into the three schedules (External, Temporal, and State), we established a clear foundation for understanding how the system should respond in each scenario.

External Events Schedule

Event Type	External Event (Trigger)	Actor	Required Use Case
External	User plays a song	User	Play Song
External	User likes or saves a song	User	Save Song
External	User updates friends list	User	Manage Friends List
External	User requests song upload	User	Request Song Upload
External	User creates, edits, or deletes a playlist	User	Manage Playlist
External	User follows another user or artist	User	Add / Follow Friend
External	Manager receives user feedback	Project Manager	Process User Feedback
External	Admin uploads a new song	Admin	Upload Song
External	Admin receives user request	Admin	Process User request

Temporal Events Schedule

Event Type	Temporal Event (Trigger)	Time/Interval	Required Use Case
Temporal	System generates user most-played report	Monthly	Generate Monthly Most-Played Report
Temporal	System generates trending songs report	Weekly	Generate Weekly Trending Report
Temporal	System sends automatic notifications	Daily / Scheduled	Send Notifications

State Events Schedule

Event Type	State Event (State Change)	State Change Description	Required Use Case
State	New follow connection created	Social graph updated	Update Recommendations
State	Songs been added to User's favorites	User statistics updated	Update User Stats & Preferences
State	Song deleted from the system	Playlists containing the song	Update Playlists / Remove References
State	System detects performance issue	Error state detected	Create Alert

Extracting Use Cases from Events: Once the event schedules were completed, we used them to derive the system's required use cases.

Each event naturally maps to a specific system response, and that response becomes a use case.

Following this method ensured that:

- every use case is grounded in a real trigger,
- no unnecessary or imaginary features were added,
- and all functional requirements are traceable back to actual system behavior.

This step created the full list of use cases used later in the detailed modeling stages.

User stories After identifying the use cases, we translated them into User Stories to describe requirements from the perspective of different actors.

To maintain clarity, we categorized the user stories into four groups based on actor type:

- Internal Operational
- External Operational
- Internal Executive
- External Executive

This classification helped align the user stories with the stakeholders they belong to and ensured that both operational and managerial needs of the system were captured.

User Stories

Internal operational

User story 1:

Error Logging

As a Developer I want the system to log errors, so that I can quickly identify and fix issues.

Acceptance criteria:

- Given an error occurs in the system, When the error happens, Then the system logs the error details
- Given the developer opens the log file, when they view the file, Then they can see the stored error information.

User story 2:

Upload Song in Database

As a System Administrator, I want to upload songs in the database, So that the users can search discover them.

Acceptance criteria:

- Given the admin selects a new song to upload, When they submit it, Then the system stores the song in the database.
- Given the song is stored, When a user searches for it, Then it appears in the search results.

External Operational

User story 3:

Create Playlist

As a user, I want to create playlists so that I can organize my favorite songs and listen to them easily.

Acceptance Criteria:

- Given I am logged in, when I create a new playlist, then it appears in my library.
- Given a playlist exists, when I add a song to it, then the playlist updates immediately.

User story 4:

Search and Browse Songs

As a user, I want to search for songs by name or artist and browse recommended or popular songs, So that I can easily find music I want to listen to

Acceptance Criteria:

- Given the user is on the search page, When the user enters a song name or artist and presses search, Then the system displays accurate and relevant search results within 2 seconds.
- Given the user is on the home or browse page, When the user selects a recommended or popular song section, Then the system displays a list of songs in that section for the user to explore.
- Given no results match the search query, When the user searches, Then the system displays a "No results found" message with suggestions.

User story 5:

As a user, I want to like a song, so that I can save it to my favorites.

Acceptance Criteria:

- Given I click the "like" button on a song, when I confirm, then the song is added to my favorites.
- Given I unlike a song, when I click "unlike," then it is removed from my favorites.

Internal Executive

User story 6: Project Manager

As a **Project Manager**, I want the system to collect and store user feedback, so that I can review suggestions and issues to improve the app.

Acceptance Criteria:

- **Given** a user submits feedback, **When** the submission is received, **Then** the system saves the feedback in the database with the user ID and timestamp.
- **Given** feedback exists in the system, **When** the Project Manager accesses the feedback section, **Then** the system displays a list of all feedback entries with details (user, message, date).

User Story 6: App Owner

As an **App Owner**, I want to analyze user and song trends so that I can make informed business decisions.

Acceptance Criteria:

- **Given** I access the insights section, **when** I select “Top Genres,” **then** I should see the most-played music categories.
- **Given** new data is generated, **when** the report is refreshed, **then** updated statistics should appear.

External Executive

Record Labels

As a Music Content Provider, I want the system to track and record song play counts, so that I can analyze listener engagement and ensure accurate royalty reporting.

Acceptance Criteria:

- **Given** a user plays a track, **When** the playback is registered, **Then** the system increments the play count for that track.
- **Given** multiple tracks have accumulated play counts, **When** the system generates statistics, **then** it produces a “Most Played” list based on the play counts.

User Story 8:

App Store Platforms (Google Play / Apple Store)

As an App Store Platform, I want the system to enforce ethical content guidelines, so that inappropriate or illegal content is not accessible to users.

Acceptance Criteria:

- **Given** a user uploads or attempts to play a track, **When** the content is flagged as inappropriate or violates guidelines, **Then** the system blocks access and marks the content for review.
- **Given** new content is added to the system, **When** it passes automated content checks, **Then** the system allows it to be played by users.

Creating Brief Use Case Descriptions

Next, we prepared Brief Use Case Descriptions for each use case derived from the events.

This step provided a short, consistent explanation of what each use case does without going into full procedural details.

These descriptions created a smooth transition between the user stories and the later UML modeling stage.

Brief Use Case Descriptions

Use Case	Brief Use Case Description
Play Song	User selects a song to play, and the system updates the user's play history
Search for a song	User searches for a song by title, artist, or genre, and the system returns matching results
Save Song	User saves or likes a song, and the system adds it to the user's favorites and updates preference data.
Manage Friends List	User updates their friends list, and the system adds or removes the selected users from the social graph.
Request Song Upload	User submits a request to upload a song, and the system records and forwards the request for review
Create / delete Playlist	User creates or deletes a playlist, and the system updates the playlist accordingly.
Add / delete songs to the playlists	User Adds or deletes a playlist, and the system updates the playlists.
Send feedbacks	User sends feedback or rates their experience, and the system stores the feedback for managerial review
Follow user / artist	User follows another user or artist, and the system updates their social connections.
Process User Feedback	Manager reviews user-submitted feedback, and the system records actions or responses taken

Receive requests	Admin receive songs requested from the User
Upload Song	Admin uploads a new song, and the system validates, stores, and publishes it to the music catalog.
Generate Monthly Most-Played Report	System analyzes monthly listening data and generates the user's most-played songs report.
Generate Weekly Trending Report	System analyzes weekly platform-wide activity and generates a trending songs report.
Send Notifications	System sends scheduled notifications to users based on activity and preferences.
Update Recommendations	System recalculates and updates recommendations when a new follow connection is created.
Update User Stats & Preferences	System updates the user's statistics and preference profile when songs are added to favorites.
Create Alert	System detects a performance issue and automatically generates an alert for admins
Update Playlists After Song Deletion	System removes a deleted song from all playlists and updates the affected playlist data.

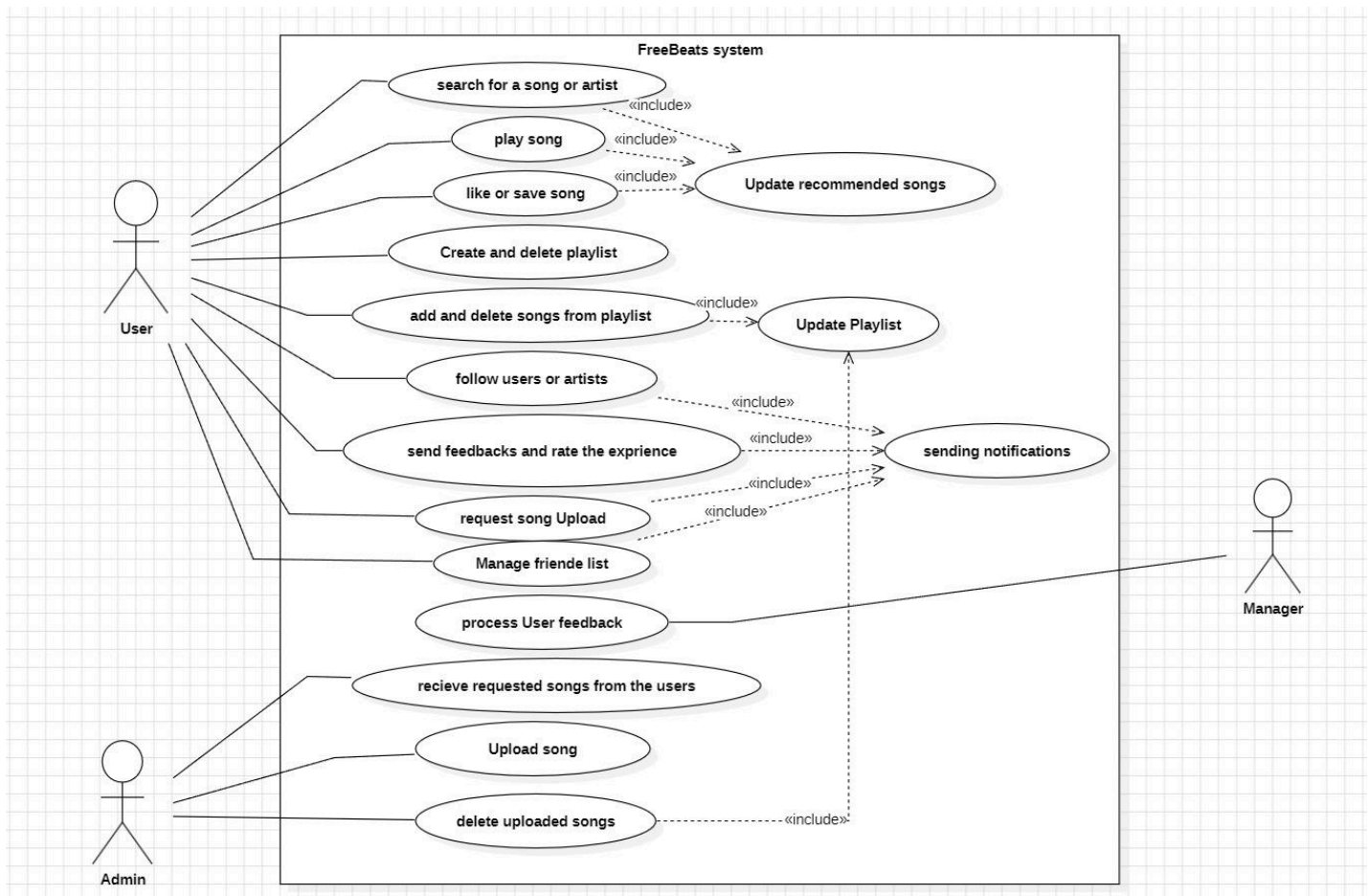
UML Use Case Diagram

After finalizing the use cases and their descriptions, we constructed the UML Use Case Diagram.

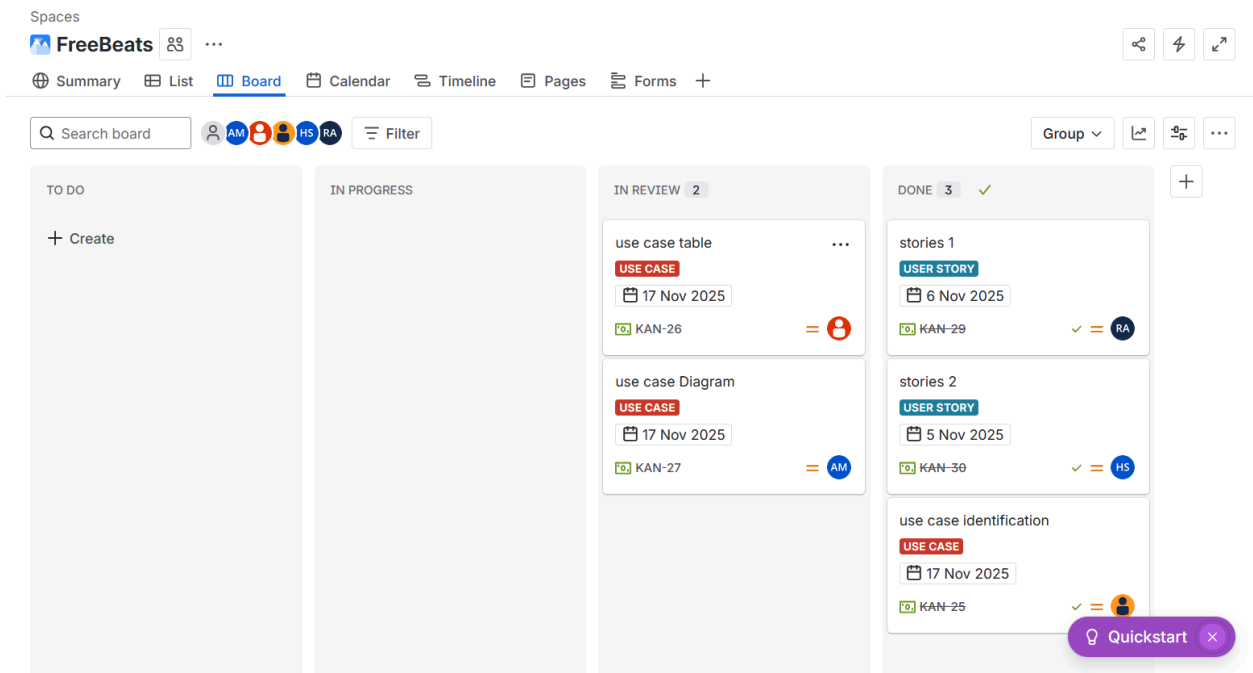
This diagram visually summarizes:

- the actors involved,
- the interactions between each actor and the system,
- and the complete scope of the functional system behavior.

It serves as a visual confirmation that all events, use cases, and user stories are aligned and represented correctly.



JIRA & GITHUB 👍😡



Spaces

...

Summary

List

Board

Calendar

Timeline

Pages

Forms

+

Q Search list

Filter

Group

...

Type	Summary	Status	Due date	Priority	Assignee	Comments	
	system vision	DONE	25 Oct 2025	Medium		Add comment	
	workflow documentation	IN PROGRESS	17 Nov 2025	Medium	23-101130	Add comment	
	Questionnaire&Agendas	DONE	25 Oct 2025	Medium		Add comment	
	UML	IN REVIEW	17 Nov 2025	Medium		Add comment	
	Documentations	DONE		Medium		Add comment	
	use case	TO DO	17 Nov 2025	Medium		Add comment	
	user story	TO DO	17 Nov 2025	Medium		Add comment	

+ Create

GITHUB 😊

<https://github.com/arwa882005/FreeBeats>

 Abdelrhman150 Submission 2	08c2eed · 1 hour ago	
 Brief Use Case Descriptions FRE...	Submission 2	1 hour ago
 Event_Schedules_FreeBeatsF.pdf	Submission 2	1 hour ago
 Functional Requirements.pdf	Add files via upload	3 weeks ago
 Interview Agenda Executive stak...	Rename Interview Agendas-executive ...	3 weeks ago
 Interview Agenda operational st...	Rename FreeBeats_Interview_Agendas ...	3 weeks ago
 Questionnaire.pdf	Adding Questionnaire	3 weeks ago
 Requirements Elicitation Techni...	Add files via upload	3 weeks ago
 Stakeholder identification.pdf	Add files via upload	3 weeks ago
 System vision document.pdf	Add files via upload	3 weeks ago
 Techincal report ffinal.pdf	Submission 2	1 hour ago